

Лабораторна робота №9. Створення інтерактивного DOM та обробка подій: Event Delegation, форми та доступність динамічного інтерфейсу

Термін здачі: 16.10.2026

1. Мета роботи

Навчитися будувати інтерактивний інтерфейс на DOM API без фреймворків: створювати елементи з шаблону, обробляти події через делегування, працювати з формами й модальним діалогом і керувати фокусом так, щоб інтерфейсом можна було користуватися з клавіатури та скрінрідера. Навчитися перевіряти JavaScript-код, написаний AI-агентом, за специфікацією й поведінкою в браузері, а не за тим, що він «начебто працює».

Очікуваний результат: застосунок «Мій список», кожную частину якого спершу написав AI-агент, а ви знайшли розбіжності з ТЗ і виправили їх власноруч; «Пройдено: 33 з 33» у самоперевірці й таблиця аудиту, де кожна знахідка має правило ТЗ, доказ і коміт.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Знаходити елементи через `querySelector*`, розрізняти живі й статичні колекції; пам'ятати, що значення `dataset` — рядки.
- Створювати елементи з `<template>`, записувати дані через `textContent` і атрибути; пояснювати, чому `innerHTML` з даними — це XSS.
- Пояснювати фази `capturing` і `bubbling`, різницю між `target` і `currentTarget`, значення `this` в обробнику; використовувати `closest()`.
- Застосовувати `event delegation` і пояснювати, чому обробник прив'язаний до вузла, а не до селектора.
- Обробляти форми: `submit` і `preventDefault()`, надсилання клавішею Enter, `input` і `change`, `reset`, `valueAsNumber`, `Constraint Validation API`.

- Керувати фокусом: `focus()`, `tabindex`, фокус після видалення елемента, модальний `<dialog>`.
- Передавати стан допоміжним технологіям: `aria-pressed`, `aria-invalid`, `aria-describedby`, `live region (role="status")`.
- Перевіряти AI-згенерований код за ТЗ, самоперевіркою й DevTools і фіксувати аудит у Git.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4) і провести експерименти: для кожного записати власний прогноз і прогноз AI-агента **до** запуску.
2. Завантажити [starter-проєкт «Мій список»](#), зафіксувати його в Git без змін і переконатися, що самоперевірка показує «Пройдено: 0 з 33» (§5.1).
3. Виконати Challenges 1–5 (§5.3–5.7): для кожного AI-агент пише чернетку, ви фіксуєте її, перевіряєте за ТЗ і виправляєте власноруч.
4. Пройти ручний чек-лист ТЗ, опублікувати проєкт на GitHub Pages, оформити звіт (§6) і захистити роботу (§7).

Важливо! Дані й модуль стану готові, тож код ЛР-8 не потрібен. Таймерів, Promise і fetch у роботі немає: Event Loop — тема Лекції 6 і ЛР-10.

4. Дослідницькі завдання

Перед практичною частиною опрацюйте джерела й проведіть експерименти. Загальна довідка — [Introduction to the DOM — MDN](#); вимоги доступності — [WCAG 2.2](#) і [ARIA Authoring Practices Guide](#); поведінку браузера визначає [HTML Living Standard](#).

Протокол і таблиця — як у ЛР-8: власний прогноз, прогноз AI без запуску, результат і пояснення хибного прогнозу. Таблиця з усіма 15 експериментами входить у звіт.

Фрагменти виконуйте в Console на сторінці `experiments.html` зі starter-проєкту (через Live Preview), оновлюючи її перед кожним експериментом. Біля кожного вказано дефект, до якого ця поведінка призводить у реальному коді.

4.1. DOM API

Опрацювати:

- [Locating DOM elements using selectors — MDN](#)
- [HTMLCollection](#) і [NodeList](#) — MDN
- [Node: textContent](#) і [Element: innerHTML](#), розділ Security considerations — MDN

- [<template>](#), [Element: replaceChildren\(\)](#), [HTMLElement: dataset](#) — MDN

Дослідити: що повертають `querySelector` і `querySelectorAll`, якщо нічого не знайдено; живі й статичні колекції; `textContent`, `innerHTML` і `innerText`; чому вміст `<template>` інертний; що робить `cloneNode(true)`.

Експерименти:

1. Видалення класу в циклі:

```
const items = document.getElementsByClassName('item');
for (let i = 0; i < items.length; i++) items[i].classList.remove('item');
document.querySelectorAll('.item').length;
```

Потім оновіть сторінку й повторіть фрагмент з `document.querySelectorAll('.item')` у першому рядку.

Дефект: цикл по живій колекції пропускає кожен другий елемент.

2. Один рядок чотирма способами:

```
const payload = (via) => `
```

Які рядки виведе Console і що з'явиться на сторінці?

Дефект: «очищення» HTML від тегів через тимчасовий `<div>` саме по собі є XSS: обробник виконується, хоча `<div>` навіть не в документі.

3. dataset:

```
const li = document.querySelector('#exp-3 li');
[li.dataset.id, typeof li.dataset.id, li.dataset.id === 82,
Number(li.dataset.id) === 82];
```

Дефект: запис за `id` з `dataset` ніколи не знаходиться.

4. innerHTML +=:

```
const box = document.querySelector('#exp-4');
box.querySelector('button').addEventListener('click', () =>
console.log('клік'));
box.querySelector('button').click();
box.innerHTML += '<p>Новий рядок</p>';
box.querySelector('button').click();
```

Дефект: додавання одного рядка мовчки ламає всі наявні кнопки.

4.2. Події та event delegation

Опрацювати:

- [Introduction to events](#) і [Event bubbling](#), зокрема розділ Event delegation — MDN
- [EventTarget: addEventListener\(\)](#) — MDN, зокрема розділи про this і про повторну реєстрацію
- [Event: target](#), [Event: currentTarget](#), [Element: closest\(\)](#) — MDN
- [Button pattern — ARIA APG](#)

Дослідити: три фази поширення події; target і currentTarget; чому this у function-обробнику — елемент, а в arrow function — ні; що повертає closest(), якщо предка немає; що буде, якщо додати обробник двічі; чому делегований обробник працює для елементів, створених пізніше; що <button> уміє порівняно з <div> з обробником click; що означає isTrusted.

Експерименти:

5. Фази:

```
const ul = document.querySelector('#exp-5 ul');
for (const el of [ul, ul.querySelector('li'), ul.querySelector('button')])
{
  const name = el.tagName.toLowerCase();
  el.addEventListener('click', () => console.log(name, 'capture'), {
capture: true });
  el.addEventListener('click', () => console.log(name, 'bubble'));
}
```

Потім клікніть кнопку мишею. Прогнозуйте порядок шести рядків.

6. Клік по іконці:

```
const button = document.querySelector('#exp-6 button');
button.addEventListener('click', function (event) {
  console.log('function:', event.target.tagName,
event.currentTarget.tagName, this.tagName);
});
button.addEventListener('click', (event) => {
  console.log('arrow:', event.target.tagName, event.currentTarget ===
button, this === window);
});
document.querySelector('#exp-6').addEventListener('click', (event) => {
  console.log('closest:', event.target.closest('button'));
});
```

Клікніть мишею по зірці, потім по тексту кнопки, потім по заголовку розділу.

Дефект: обробник читає event.target.dataset, і клік по іконці нічого не робить.

7. Повторна реєстрація:

```
const button = document.querySelector('#exp-7 button');
let count = 0;
const named = () => count++;
button.addEventListener('click', () => count++);
button.addEventListener('click', () => count++);
button.addEventListener('click', named);
button.addEventListener('click', named);
button.click();
count;
```

Дефект: обробник, доданий у функції перемальовування, після кожного перемальовування спрацьовує на один раз більше.

8. <div> чи <button>:

```
const div = document.querySelector('#exp-8 .fake');
const button = document.querySelector('#exp-8 button');
div.addEventListener('click', (event) => console.log('div',
event.isTrusted));
button.addEventListener('click', (event) => console.log('button',
event.isTrusted));
```

Клікніть обидва елементи мишею. Чи потрапляє на <div> фокус клавішею Tab? На кнопки натисніть Enter і Space. Потім виконайте `div.click()`; `button.click()`;

Дефект: «кнопкою» на <div> неможливо скористатися з клавіатури (WCAG 2.1.1).

4.3. Форми

Опрацювати:

- [Client-side form validation — MDN](#)
- [Constraint validation API](#) і [HTMLInputElement: setCustomValidity\(\)](#) — MDN
- [HTMLFormElement: submit event](#), [reset event](#), [requestSubmit\(\)](#) — MDN
- [Element: input event](#) і [change event](#) — MDN
- [Implicit submission — HTML Living Standard](#)

Дослідити: коли форму надсилає Enter і що станеться без `preventDefault()`; чим `submit` відрізняється від `click` по кнопці; коли настають `input` і `change`; коли настає `reset` відносно очищення полів; чим `value` відрізняється від `valueAsNumber`; що таке `validity`, `checkValidity()` і `setCustomValidity()`; що робить атрибут `novalidate`.

Експерименти:

9. Увімкніть у Console **Preserve log** і виконайте:

```
const form = document.querySelector('#exp-9 form');
form.querySelector('button').addEventListener('click', (event) =>
console.log('click', event.isTrusted));
form.addEventListener('submit', () => console.log('submit'));
```

Клікніть у поле, введіть drama і натисніть Enter. Які події настали і що сталося зі сторінкою та її адресою?

Дефект: Enter у полі пошуку перезавантажує сторінку, і стан застосунку втрачено.

10. Числове поле:

```
const input = document.querySelector('#exp-10 input');
input.value = '7';
const filled = [input.value + 1, input.valueAsNumber + 1];
input.value = '';
[filled, [input.value + 1, input.valueAsNumber + 1]];
```

Дефект: оцінка з поля форми — рядок, і rating + 1 дає '71'.

11. Власне повідомлення:

```
const input = document.querySelector('#exp-11 input');
input.value = '42';
if (input.valueAsNumber > 10) input.setCustomValidity('Не більше 10');
input.value = '5';
[input.validity.valid, input.validity.customError, input.checkValidity(),
input.validationMessage];
```

Дефект: після першої помилки форма лишається невалідною назавжди.

12. Подія reset:

```
const form = document.querySelector('#exp-12 form');
form.addEventListener('reset', () => console.log('y reset:',
JSON.stringify(form.elements.q.value)));
form.elements.q.value = 'drama';
form.reset();
console.log('після reset():', JSON.stringify(form.elements.q.value));
```

Потім введіть у поле щось інше й натисніть «Скинути» мишею.

Дефект: «Скинути» очищає поля, а список перемальовано зі старим фільтром.

4.4. Фокус, діалоги й доступність

Опрацювати:

- [HTMLElement: focus\(\)](#) і [tabindex](#) — MDN
- [<dialog>](#) і [HTMLElement: inert](#) — MDN; [Dialog_\(Modal\)_pattern](#) — ARIA APG
- [ARIA live regions](#), [aria-pressed](#), [aria-invalid](#), [aria-describedby](#) — MDN
- WCAG 2.2: [2.1.1 Keyboard](#), [2.4.3 Focus Order](#), [2.4.7 Focus Visible](#), [2.5.3 Label in Name](#), [3.3.1 Error Identification](#), [4.1.2 Name, Role, Value](#), [4.1.3 Status Messages](#)

Дослідити: які елементи отримують фокус без `tabindex`; `tabindex="-1"` і `tabindex="0"`; куди переходить фокус, коли елемент із фокусом видалено; що робить `showModal()` з рештою сторінки і куди повертає фокус `close()`; що таке `accessible name` і звідки його бере браузер; чому `live region` має бути в документі до появи тексту. Почути оголошення можна лише скрінрідером (Narrator, NVDA, VoiceOver) — це необов'язково.

Для експериментів 13–15 додайте в Console **Live Expression** `document.activeElement`.

Експерименти:

13. Видалення елемента з фокусом:

```
const button = document.querySelector('#exp-13 button');
button.focus();
const before = document.activeElement === button;
button.remove();
[before, document.activeElement];
```

Дефект: після перемальовування списку користувач клавіатури опиняється на початку сторінки.

14. `focus()` і `tabindex`:

```
const box = document.querySelector('#exp-14 div');
box.focus();
const plain = document.activeElement === box;
box.tabIndex = -1;
box.focus();
[plain, document.activeElement === box];
```

Потім натисніть Tab на кнопці з експерименту 13: чи потрапляє фокус на цей `<div>`?

15. Модальний діалог:

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
const outside = section.querySelector('.outside');
opener.focus();
dialog.showModal();
outside.focus();
console.log('після outside.focus():',
document.activeElement.textContent.trim());
dialog.close();
console.log('після close():', document.activeElement.textContent.trim());
```

Оновіть сторінку й виконайте варіант, у якому кнопку, що відкрила діалог, видалено, поки діалог відкритий. Куди перейшов фокус (дивіться Live Expression)?

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
opener.focus();
dialog.showModal();
opener.remove();
dialog.close();
```

Дефект: після збереження картку перемальовано, і фокус не повертається нікуди.

4.5. DOM-код, написаний AI

Дослідити, що AI-агент зазвичай робить у DOM-кодi, якщо його про це не просили:

- будує розмітку через `innerHTML` і шаблонний рядок із даними;
- додає обробник на кожну картку — один раз на старті або щоразу під час перемальовування;
- читає `event.target` без `closest()`;
- не скасовує надсилання форми;
- створює рядок статусу заново або не створює зовсім;
- вставляє опис через `innerHTML` або «очищує» його через тимчасовий `<div>`;
- робить модальне вікно з `<div>`;
- показує помилки валідації через `alert()`;
- губить фокус після перемальовування;
- додає зайвий `ARIA: role="button"` на `<div>` без підтримки клавіатури, `aria-label`, що підмінює видимий текст (WCAG 2.5.3).

Для кожного пункту запишіть, яке правило ТЗ він порушує і як його помітити.

5. Практична частина: AI-чернетка → аудит → виправлення

Starter-проект «Мій список» — каталог 30 серіалів [Tvmaze](#) з вибірки ЛР-8. HTML, CSS, дані й модуль стану `src/state.js` готові; функції шару DOM у `src/` порожні. Як має працювати застосунок, описано в [ТЗ](#): правила пронумеровано (C2.4 — четверте правило Challenge 2), і `check.html` перевіряє кожне під тим самим номером. Виконуйте challenges по черзі.

5.1. Підготовка

1. Завантажте [watchlist-starter.zip](#) і розпакуйте в **окремий репозиторій** або в теку `lab-9/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**.
2. Відкрийте `index.html` через Live Preview: форма фільтрів, порожній список, Console без помилок.
3. Відкрийте так само `check.html`: «Пройдено: 0 з 33». Прочитайте, чого перевірка не бачить.
4. `check.html`, `check/`, `data/` і `src/state.js` не змінюються в жодному коміті.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** Як у ЛР-8: AI дає прогноз **до** запуску, і його відповідь записано так, як він її дав.
- **Challenges.** Кожен проходить чотири кроки:
 1. **Чернетка.** Дайте AI-агенту в IDE (режим, у якому він сам змінює файли проекту) **лише** запит challenge, без ТЗ, без `check.html` і без власних уточнень.
 2. **Коміт без змін**, навіть якщо чернетка не працює: C2: AI draft.
 3. **Аудит.** Запустіть `check.html` і пункти ручного чек-листа ТЗ, що стосуються challenge. Кожну розбіжність запишіть у таблицю аудиту (§5.8).
 4. **Виправлення власноруч.** AI може пояснити перевірку, правило чи помилку, але **не пише** код виправлення. Коміт: `fix(C2): <що виправлено>`, один або кілька.

Чернетка, що з першого разу пройшла все, — теж результат: запишіть в аудит, як ви її перевірили.

Приклад запиту для пояснення (режим без редагування — `ask / plan`):

Поясни, чому падає ця перевірка. Нижче — її назва й повідомлення та мій код. Не пиши виправлений код: назви механізм DOM або правило доступності, яке я, ймовірно, не врахував, і як перевірити це в DevTools.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Challenge 1 — картки

Покажи серіали з `data/shows.json` у списку `#results`: назва, рік, жанри, оцінка, кнопки «До списку» і «Детальніше».

Правила T3 C1.1–C1.7. Теми: §4.1. Файл: `src/render.js`.

5.4. Challenge 2 — кнопки карток

Зроби, щоб «До списку» додавав серіал у мій список, а «Детальніше» відкривав опис.

Правила C2.1–C2.7. Теми: §4.2, §4.4. Файл: `src/list.js`.

5.5. Challenge 3 — фільтри

Додай фільтрацію за назвою і жанром і перемикач «лише мій список».

Правила C3.1–C3.7. Теми: §4.3, §4.4. Файли: `src/filters.js`, для C3.7 — `src/list.js`.

5.6. Challenge 4 — діалог «Детальніше»

Зроби діалог «Детальніше» з описом серіалу.

Правила C4.1–C4.5. Теми: §4.1, §4.4. Файл: `src/details.js`.

5.7. Challenge 5 — моя оцінка й нотатка

Додай у діалог форму «моя оцінка й нотатка» з валідацією.

Правила C5.1–C5.7. Теми: §4.3, §4.4. Файл: `src/details.js`.

5.8. Аудит і коміти

Аудит — таблиця, щонайменше один рядок на challenge, і вона входить у звіт:

Challenge	Правило ТЗ	Що було в AI-чернетці	Доказ (check / DevTools / клавіатура)	Виправлення (коміт)
C2	C2.4	<code>event.target.dataset.action</code> без <code>closest()</code>	X C2.4: клік по іконці отримав <code><svg></code>	<code>fix(C2): find the button with closest()</code>

Коміти кожного challenge — C<N>: AI draft, потім fix(C<N>): ..., тож git diff між ними і є аудитом. Після C5 пройдіть ручний чек-лист T3 і опублікуйте проєкт на GitHub Pages (інструкція — у README.md).

6. Вимоги до звіту

Тема листа:

[Веб] ЛР-9: Створення інтерактивного DOM та обробка подій: Event Delegation, форми та доступність динамічного інтерфейсу

У листі вкажіть:

GitHub repository: `https://github.com/<username>/<repository>`
GitHub Pages: `https://<username>.github.io/<repository>/`

За адресою Pages викладач відкриє і застосунок, і `check.html`.

Звіт повинен містити:

1. таблицю експериментів §4 (усі 15) і короткі висновки до кожного підрозділу;
2. назву та версію AI-агента, модель, дату роботи;
3. таблицю аудиту (§5.8) для всіх п'яти challenges;
4. результат `check.html` на опублікованій сторінці — «Пройдено: 33 з 33»;
5. ручний чек-лист T3 з результатом кожного пункту;
6. підсумок: що AI зробив правильно без підказки і чого не зробив жодного разу, доки його не попросили.

7. Усний захист

Формат задачі передбачає усну співбесіду за результатами практичної реалізації. Виправлення пиште ви, тож на захисті змінюєте код наживо, без AI, і відповідаєте на питання з демонстрацією екрана.

Орієнтовні питання:

Аудит

- Покажіть одну знахідку в AI-чернетці: перевірку, що її виявила, причину й коміт, який її виправив.
- Змініть код наживо: кнопка «Прибрати нотатку» на картці через наявний делегований обробник, фокус на тексті нотатки після збереження або `<select>` сортування у формі фільтрів.

Події

- Чому клік по іконці має інший `target`? Що повертає `closest()` для кліку поза кнопками?
- Чому картка, створена після запуску, працює з делегуванням, але не з обробниками на кожній картці?

Форми й фокус

- Чому Enter у полі пошуку перезавантажує сторінку, якщо `submit` ніхто не обробляє?
- Куди переходить фокус, коли елемент із фокусом видалено? Що у вашому коді цьому запобігає?

DOM і безпека

- Чому для нотатки достатньо `textContent`? Що не так із `div.innerHTML = summary;`
`div.textContent`?

AI

- Чого AI не зробив жодного разу, доки його не попросили? Чому, на вашу думку?

8. Оцінювання

1. Успішний захист — **2 бали**.
2. Додатковий **1 бал** за однієї з умов:
 - термін здачі до **16.10.2026**, АБО
 - робота на парі.